

STAFF FIREBALL MAGIC

A complete 3DG Engine workflow: Character Editor staff attachment, action timing, Particle Editor effects, runtime prototypes, collision, damage, scripting, Play-mode testing, and standalone export.

Project-aware edition

Built around your existing Character asset, StaffSocket on hand_r, StaffAttack action clip, and PlayerAttack script.

3DG Engine developer guide

Prepared from the engine code as of 24 July 2026

Contents

1. Result and architecture
2. Prepare the Content folders
3. Verify the character, staff, and attack action
4. Add the ReleaseFireball animation event
5. Build FireballTrail in the Particle Editor
6. Build FireballImpact in the Particle Editor
7. Create the runtime prototype objects
8. Extend PlayerAttack
9. Attach, compile, and test
10. Troubleshooting and polish

1. Result and architecture

Press left mouse or F. PlayerAttack accepts the input only when StaffAttack is not already playing and its recovery timer has finished. The action plays, an animation event marks the release frame, and the script reads the animated first staff attachment to calculate the exact world-space tip. A cloned fireball prototype starts its trail and travels as a CCD kinematic trigger. On impact it damages Health, spawns a one-shot burst, and cleans both effects up.



Why the projectile logic stays in PlayerAttack

SpawnFromObject clones ordinary components such as Transform, Collider, Audio Source, and Particle System. NativeScriptComponent owns a unique runtime instance and is not copy-constructible, so it is intentionally skipped by Registry::Clone. Tracking spawned fireballs in PlayerAttack avoids relying on a script component being cloned.

Setting	Value	Purpose
Character	Character.3dgcharacter	Existing animated player character.
Staff socket	StaffSocket / hand_r	Existing attachment point in the Character Editor.
Action	StaffAttack	Existing standalone action clip called from PlayerAttack.
Release event	ReleaseFireball	New event at the frame where magic leaves the staff.
Trail	FireballTrail.particle	Continuous additive flame around the moving projectile.
Impact	FireballImpact.particle	One-shot additive burst on collision or range expiry.

2. Prepare the Content folders

Use the Assets panel so all authoring stays inside the editor. Under the project's Content folder, create the following structure. The names are recommendations; the prototype object names used by the script are exact.

```
Content/  
  Assets/  
    Characters/  
      AnimationActionClips/  
      AnimationGraphs/  
      Particles/  
      Audio/  
      Materials/  
  Scripts/  
    PlayerAttack.h  
  Scenes/
```

- Particle assets: save reusable .particle files under Assets/Particles.
- Audio: optional cast and impact WAV/OGG files go under Assets/Audio.
- Scripts: keep PlayerAttack.h under Content/Scripts so the editor compiler and standalone build use the same source.
- Scene: save before Play and export after the final test.

Existing project check

Your current project already references Content/Scripts/PlayerAttack.h from the Character asset and auto-registers PlayerAttack through EditorGeneratedScripts.h. You do not need a second caster class.

3. Verify the character, staff, and attack action

1. Open Panels > Character Editor and load Character.3dgcharacter.
2. Open Mesh. Confirm the skeletal model and material are loaded.
3. In Sockets, confirm StaffSocket targets hand_r. Your current socket position is (-14.02, -4.24, 0) with rotation (17, 0, 0).
4. In Attachments, make the magic staff the first attachment and select StaffSocket. The script intentionally reads attachments.front().
5. Open Animation. Confirm the locomotion graph is PlayerGraph.3dggraph and StaffAttack.3dgclip is listed under Standalone Action Clips.
6. Open Script. Confirm PlayerAttack is enabled, then Apply to Selected or Add to Scene.

Do not add the staff as a separate scene object

The Character Editor attachment follows the animated hand bone and already accounts for the model's import orientation. The script reads the same resolved attachment matrices, so the projectile begins where the rendered staff is.

4. Add the ReleaseFireball animation event

The cast should release on the animation frame, not immediately on button press. The engine forwards authored animation events to scripts through `WasAnimationEvent()`.

1. Place or select the player character in the scene.
2. Open Panels > Animation Preview. Select the StaffAttack clip from the clip list.
3. Play or scrub the clip to the frame where the staff points toward the target and the spell should leave the tip.
4. Expand Events and choose Add Event. The panel uses the current preview time.
5. Set Clip to StaffAttack's clip index, set Name exactly to ReleaseFireball, and fine-tune Time if needed.
6. Save the scene. To preserve the event in the reusable character asset, open Character Editor, choose Capture Selected, then Save the Character asset.

Event names are case-sensitive

ReleaseFireball, releasefireball, and Release Fireball are different strings. The script and Animation Preview must use the exact same spelling.

How action timing and anti-spam work

- Input is rejected while `IsAnimationActionPlaying()` is true.
- A 0.65 second recovery timer rejects rapid clicks after an accepted cast.
- `PlayActionClip("StaffAttack")` queues a cast only if the saved Action Name is found.
- The projectile is created only when ReleaseFireball is received.
- If the action is interrupted before the event, the pending cast is cancelled.

Calibrate StaffTipX/Y/Z

The attachment matrix reaches the staff object's local origin. StaffTipX, StaffTipY, and StaffTipZ add a point in the staff model's local coordinates. Begin with (0, 0, 1). If the staff mesh uses a different axis or larger source units, cast repeatedly and change one axis at a time until the fireball begins at the gem or upper end. Negative values move in the opposite local direction.

5. Build FireballTrail in the Particle Editor

1. Open Panels > Particle Editor and choose New. Name the asset FireballTrail.
2. In System Playback, enable the system, turn Autoplay off, enable Loop, disable Prewarm, and enable Local Space off. The script starts cloned instances with RestartParticles.
3. Use the module stack. Select required modules or choose Add Module for optional modules.
4. Use the isolated preview's Play/Restart controls while tuning. Enable bloom/post processing in the scene to see HDR additive color correctly.
5. Save as Content/Assets/Particles/FireballTrail.particle.

Setting	Value	Purpose
System: Duration	2.0 s	Longer than a normal flight; looping keeps emission active.
System: Local Space	Off	Old particles remain behind and form a trail.
System: Backend	Auto	Uses GPU compute when compatible; CPU fallback remains valid.
Spawn: Rate	260 /s	Dense continuous flame without an excessive pool.
Spawn: Maximum	900	Safe upper bound for the short lifetime.
Spawn: Burst	8	Immediate core when the projectile appears.
Spawn: Lifetime	0.18 - 0.45 s	Short-lived particles form a compact tail.
Shape	Sphere, radius 0.08	Emits around the projectile centre.
Initial Velocity	0.05 - 0.45	Small randomized drift.
Forces	Gravity (0, 0.8, 0), Drag 3	Flame rises slightly and slows quickly.
Color Over Life	(4.5,1.5,0.15,1) to (0.9,0.02,0,0)	HDR orange cools and fades.
Size Over Life	0.22 to 0.02	Hot core shrinks into the tail.
Renderer	Billboard, Additive	Camera-facing fire with bright overlap.
Bounds Radius	3.0	Prevents premature culling around the moving emitter.

Optional texture

Drag a soft circular flame/smoke texture onto Renderer > Texture. Use one column and one row for a still texture, or configure Columns, Rows, and Frame Rate for a flipbook. A missing texture still produces colored billboards.

6. Build FireballImpact in the Particle Editor

Create a separate one-shot burst. Keeping trail and impact as different assets lets them be reused and tuned independently.

1. Choose New and name it FireballImpact.
2. Enabled on, Autoplay off, Loop off, Prewarm off, Local Space off, Duration 1.0 s.
3. Set continuous Rate to 0. A Burst Count supplies every particle at once.
4. Save as Content/Assets/Particles/FireballImpact.particle.

Setting	Value	Purpose
Spawn: Rate	0 /s	No continuous emission.
Spawn: Maximum	500	Enough room for the burst.
Spawn: Burst Count	110	A strong but controlled impact.
Spawn: Lifetime	0.25 - 0.80 s	Mixed fast sparks and lingering flame.
Shape	Sphere, radius 0.14	Starts around the hit point.
Initial Velocity	3.0 - 8.0	Drives the radial burst.
Forces	Gravity (0,-2.5,0), Drag 1.2	Sparks arc downward and decelerate.
Color Over Life	(5,1.8,0.2,1) to (0.8,0.01,0,0)	Hot impact fades to dark red.
Size Over Life	0.35 to 0.0	Particles vanish cleanly.
Collision (optional)	Kill, radius 0.03	Stops sparks at solid geometry.
Renderer	Billboard, Additive	Bright magical burst.
Bounds Radius	6.0	Contains the full outward expansion.

Preview checklist

- The trail starts immediately when Restart is pressed.
- The trail leaves particles behind when the preview emitter moves.
- The impact creates one burst and stops; it does not emit continuously.
- No warning appears in the Compiled Pipeline Graph.
- HDR colors remain above 1.0 and become visibly brighter with bloom.

7. Create the runtime prototype objects

FireballPrototype

1. Drag FireballTrail.particle from Assets into the scene, or add an Empty object and add a Particle System component.
2. Rename the object exactly FireballPrototype. Open the Particle Editor with this object selected and ensure it references FireballTrail.particle.
3. Add a Sphere Collider: radius about 0.16, Overlap Only/Trigger enabled.
4. Set Object Channel to Projectile. In Channel Responses, enable World Static, World Dynamic, and Enemy. Disable Player, Collectible, Trigger, and Projectile.
5. Do not add a Rigid Body. PlayerAttack creates a kinematic CCD body for each clone.
6. Optional: add a spatial Audio Source with the cast/whoosh sound, Autoplay off, Loop off.
7. Move the prototype below the level. With Autoplay off it remains silent and invisible until cloned.

FireballImpactPrototype

1. Drag FireballImpact.particle into the scene and rename the object exactly FireballImpactPrototype.
2. No collider or rigid body is required.
3. Optional: add a spatial impact Audio Source, Autoplay off, Loop off.
4. Move it below the level. PlayerAttack clones it at the collision point and deletes it after 1.25 seconds.

Prototype names are runtime keys

SpawnFromObject searches RuntimeName. If the names differ by even one character, the returned entity is kNull and nothing appears. Keep FireballPrototype and FireballImpactPrototype exact.

Collision expectations

Filtering is mutual: the projectile mask must include the target channel, and the target mask must include Projectile. Give enemy characters an Enemy channel and allow Projectile. Walls should use World Static and allow Projectile. The fireball is a trigger so it reports an event without pushing either object.

8. Extend PlayerAttack

Open Content/Scripts/PlayerAttack.h in the built-in editor or your configured external code editor. Replace the current class with the following version. It preserves the existing StaffAttack anti-spam behavior and adds staff-tip spawning, flight, collision, damage, impact particles, optional audio, range expiry, and cleanup.

```
#pragma once

#include <engine/gameplay/Script.h>
#include <engine/gameplay/GameplayComponents.h>
#include <engine/animation/AnimatedModel.h>
#include <engine/graphics/SkinnedModel.h>
#include <engine/physics/PhysicsComponents.h>
#include <engine/physics/PhysicsWorld.h>
#include <GLFW/glfw3.h>

#include <algorithm>
#include <vector>

class PlayerAttack final : public engine::Script {
public:
    void OnUpdate(float dt) override {
        m_recovery = std::max(0.0f, m_recovery - dt);

        // Release on the authored frame of StaffAttack.
        if (m_castPending && WasAnimationEvent("ReleaseFireball")) {
            SpawnFireballFromStaffTip();
            m_castPending = false;
        }
        // Do not leave a pending cast behind if the action was interrupted.
        if (m_castPending && !IsAnimationActionPlaying()) {
            m_castPending = false;
        }

        UpdateImpacts();
        UpdateExplosionLifetimes(dt);

        const bool pressed =
            WasMouseButtonPressed(GLFW_MOUSE_BUTTON_LEFT) ||
            WasKeyPressed(GLFW_KEY_F);
        if (!pressed || IsAnimationActionPlaying() || m_recovery > 0.0f) {
            return;
        }

        if (PlayActionClip("StaffAttack")) {
            m_castPending = true;
            m_recovery = 0.65f;
        }
    }

    void OnFixedUpdate(float) override {
        // Kinematic velocity moves each projectile. We only enforce range here.
        for (std::size_t i = m_fireballs.size(); i-- > 0;) {
            const auto entity = m_fireballs[i].entity;
            const auto* t = TryGet<engine::ecs::Transform>(entity);
            if (!t || !Registry()->Valid(entity)) {
                m_fireballs.erase(m_fireballs.begin() + i);
                continue;
            }
        }
    }
};
```

Code continues on the next page.


```

        }
        if (glm::length(t->position - m_fireballs[i].origin) >= 30.0f) {
            Explode(i, t->position, engine::ecs::kNull);
        }
    }
}

private:
struct ActiveFireball {
    engine::ecs::Entity entity = engine::ecs::kNull;
    glm::vec3 origin{0.0f};
};
struct TimedEffect {
    engine::ecs::Entity entity = engine::ecs::kNull;
    float remaining = 0.0f;
};

bool StaffTipWorld(glm::vec3& position) const {
    const auto* root = Transform();
    const auto* animated = TryGet<engine::AnimatedModel>();
    if (!root || !animated || !animated->model ||
        animated->attachments.empty()) {
        return false;
    }

    // Character Editor setup: make the staff the first attachment.
    const engine::ModelAttachment& staff = animated->attachments.front();
    glm::mat4 world = root->Model() * animated->renderOffset;
    if (staff.bone >= 0 &&
        staff.bone < static_cast<int>(animated->pose.size())) {
        world *= animated->pose[static_cast<std::size_t>(staff.bone)]
            * staff.boneBind;
    }
    world *= staff.localOffset;

    // Local staff-model offset. Tune these three values for your staff mesh.
    const glm::vec3 localTip(
        GetFieldFloat("StaffTipX", 0.0f),
        GetFieldFloat("StaffTipY", 0.0f),
        GetFieldFloat("StaffTipZ", 1.0f));
    position = glm::vec3(world * glm::vec4(localTip, 1.0f));
    return true;
}

void SpawnFireballFromStaffTip() {
    glm::vec3 tip;
    const auto* player = Transform();
    if (!player || !StaffTipWorld(tip)) return;

    const auto fireball = SpawnFromObject("FireballPrototype", tip);
    if (fireball == engine::ecs::kNull) return;
}

```

Code continues on the next page.

```

const glm::vec3 forward = glm::normalize(
    player->rotation * glm::vec3(0.0f, 0.0f, 1.0f));
if (auto* t = TryGet<engine::ecs::Transform>(fireball)) {
    t->position = tip;
    t->rotation = player->rotation;
}

engine::ecs::RigidBody body;
body.useGravity = false;
body.kinematic = true;
body.ccd = true;
body.freezeRotation = true;
body.velocity = forward * 14.0f;
Registry()->Add<engine::ecs::RigidBody>(fireball, body);

RestartParticles(fireball);
PlayAudio(fireball, true); // optional whoosh Audio Source on prototype
m_fireballs.push_back({fireball, tip});
}

void UpdateImpacts() {
    const auto* input = Context().input;
    if (!input || !input->physicsEvents) return;

    for (const engine::CollisionEvent& event : *input->physicsEvents) {
        if (event.phase != engine::CollisionEvent::Phase::Enter) continue;
        for (std::size_t i = m_fireballs.size(); i-- > 0;) {
            const auto fireball = m_fireballs[i].entity;
            if (event.a != fireball && event.b != fireball) continue;
            const auto other = event.a == fireball ? event.b : event.a;
            if (other == Self()) continue;
            Explode(i, event.point, other);
            return;
        }
    }
}

void Explode(std::size_t index, const glm::vec3& point,
    engine::ecs::Entity hit) {
    if (index >= m_fireballs.size()) return;
    const auto fireball = m_fireballs[index].entity;

    if (auto* health = TryGet<engine::Health>(hit)) {
        health->Damage(25.0f);
    }

    const auto burst = SpawnFromObject("FireballImpactPrototype", point);
    if (burst != engine::ecs::kNull) {
        RestartParticles(burst);
        PlayAudio(burst, true); // optional impact Audio Source
        m_impacts.push_back({burst, 1.25f});
    }
}

```

Code continues on the next page.

```
        Destroy(fireball);
        m_fireballs.erase(m_fireballs.begin() + index);
    }

    void UpdateExplosionLifetimes(float dt) {
        for (std::size_t i = m_impacts.size(); i-- > 0;) {
            m_impacts[i].remaining -= dt;
            if (m_impacts[i].remaining <= 0.0f) {
                Destroy(m_impacts[i].entity);
                m_impacts.erase(m_impacts.begin() + i);
            }
        }
    }

    float m_recovery = 0.0f;
    bool m_castPending = false;
    std::vector<ActiveFireball> m_fireballs;
    std::vector<TimedEffect> m_impacts;
};
```

9. Attach, compile, and test

1. Save PlayerAttack.h.
2. In the editor's script tools, choose Compile Scripts. Resolve all errors before entering Play. The generated registration already contains PlayerAttack in this project.
3. Select the player. Confirm Script Enabled is checked, Saved Script is PlayerAttack, and the Script Class reads PlayerAttack.
4. Use Add Field three times and create Float fields StaffTipX, StaffTipY, and StaffTipZ. Start with 0, 0, and 1.
5. Save the scene, open Panels > Game Mode Settings, select the player object, and set the desired camera override before Play.
6. Press Play. Press F or left mouse once. Confirm StaffAttack starts and no second click restarts it.
7. At ReleaseFireball, confirm the projectile appears at the staff tip, moves forward, leaves a trail, collides, damages Health, and produces one impact burst.
8. Tune StaffTipX/Y/Z in Edit mode. Re-enter Play after each change because Play uses a runtime preview copy.

Setting	Value	Purpose
Input	Left mouse / F	Both are accepted by PlayerAttack.
Action Name	StaffAttack	Must match the Action Name inside StaffAttack.3dgclip.
Event Name	ReleaseFireball	Must match the Animation Preview event.
Recovery	0.65 s	Prevents attack spam after an accepted cast.
Projectile speed	14 units/s	Set in SpawnFireballFromStaffTip.
Projectile range	30 units	Set in OnFixedUpdate.
Damage	25	Applied to engine::Health on impact.
Impact lifetime	1.25 s	Time before the cloned burst entity is destroyed.

Standalone build

After the Play-mode test succeeds, save and export the runtime scene. Compile the game scripts and build/package the player. Because PlayerAttack is generated into the shared game module, the editor preview and standalone player use the same class. Keep the exported scene and its Content assets together so particle, audio, character, graph, and action-clip paths resolve.

10. Troubleshooting and polish

Symptom	Corrective action
Nothing spawns	Check exact prototype names and ReleaseFireball spelling. Confirm the particle assets are assigned and the event uses StaffAttack's runtime clip index.
Attack plays but no event	Open Animation Preview, select StaffAttack, verify the event time is within the clip, then save the scene.
Fireball starts at the hand	Tune StaffTipX/Y/Z. The values are in the staff model's local coordinates after the StaffSocket transform.
Fireball starts at the wrong staff	Make the casting staff the first Character Editor attachment, or change attachments.front() to the intended index.
Fireball flies backward	Change local forward from +Z to -Z in SpawnFireballFromStaffTip.
Fireball crosses walls	Increase collider radius, confirm both masks allow the collision, retain Rigidbody CCD, and avoid extremely high speeds.
Fireball hits the player	Projectile must ignore Player and the player's mask should ignore Projectile.
Trail follows as one blob	Turn Local Space off in FireballTrail.
Impact repeats	Impact Rate must be 0, Loop off, and Burst Count nonzero.
Particles disappear near camera edge	Increase Renderer Bounds Radius.
Colors look dull	Use Additive blend, HDR values above 1, and enable bloom/post processing.
Damage does nothing	The target needs engine::Health and must receive the projectile trigger event.
Standalone has no script	Compile scripts, rebuild the game/player, and verify PlayerAttack remains in EditorGeneratedScripts.h.

Polish ideas

- Charge: hold the button, scale speed/damage and increase the trail rate with SetParticleRate.
- Camera feedback: call ShakeCameraAdvanced on impact for a close-range hit.
- Staff charge glow: create a second particle prototype that follows the staff attachment while m_castPending is true.
- Element variants: duplicate the assets for ice, poison, arcane, or lightning and select prototypes by a String script field.
- Area damage: on impact, iterate Transform + Health targets within a radius and apply falloff.
- Homing: update the kinematic body's velocity toward a chosen target each fixed step.
- Pooling: replace clone/destroy with a reusable projectile pool when hundreds of spells can be active.

Completion checklist

- Staff is first attachment and follows StaffSocket on hand_r.
- StaffAttack exists as a standalone action and cannot be spammed.
- ReleaseFireball is authored on the correct clip and release time.
- Trail and impact assets save and preview without pipeline errors.
- Both runtime prototypes have exact names and Autoplay off.
- Projectile collision filters ignore Player and accept Enemy/world channels.
- PlayerAttack compiles, is enabled, and is registered.
- Staff-tip fields are calibrated on the chosen staff model.
- Play mode and standalone export both produce the same spell.

End of guide. The complete spell now originates from the animated staff tip, uses the Particle Editor from authoring through impact, and remains controlled by the existing PlayerAttack script and action-clip anti-spam rules.